ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN-ĐIỆN TỬ



SINGLE-CYCLE RV32I PROCESSOR

MILESTONE 2

GVHD: TRẦN HOÀNG LINH
Sinh viên: HỒ NGỌC CƯỜNG 2570082
NGUYỄN XUÂN NAM 2570462

TP. Hồ Chí Minh, Tháng 10/2026

Mục lục

Tóm tắt **1**

1 Mục tiêu và yêu cầu **1**

2 Cấu trúc project và môi trường **1**

3 Kiến trúc processor **2**
3.1 Tổng quan kiến trúc single-cycle . . . 2
3.2 Sơ đồ khối và đường đi dữ liệu . . . 2
3.2.1 Đường đi dữ liệu trong một chu kỳ . . . 2
3.3 Các khối chức năng chính . . . 2
3.3.1 ALU và register file . . . 2
3.3.2 Branch comparison unit . . . 2
3.3.3 Controller và immediate generator . . . 3
3.3.4 Load/store unit . . . 3
3.3.5 Instruction memory và data memory . . . 3
3.4 Tổ chức mã nguồn RTL . . . 3
3.4.1 Tổ chức mã nguồn . . . 3
3.4.2 Điều phối và kiểm tra tính hợp lệ . . . 4
3.4.3 Sơ đồ khối tổng thể . . . 4
3.4.4 Sơ đồ wrapper và giao tiếp MMIO . . . 5
3.4.5 Interface của core . . . 5
3.5 Testbench và phương pháp kiểm tra . . . 6

4 Quy trình simulation **6**

5 Kết quả simulation **6**
5.1 ISA regression . . . 6
5.2 Tower of Hanoi . . . 7
5.3 PROG_SORT: Bubble sort và các thuật toán sắp xếp . . . 7
5.4 PROG_MATRIX: Nhân ma trận . . . 8
5.5 PROG_SOFTFLOAT: Software floating-point . . . 8
5.6 PROG_SIGMOID: Sigmoid và classifier . . . 9
5.7 PROG_CONVOLUTION: Convolution hai chiều . . . 9
5.8 Tổng hợp kết quả các chương trình C . . . 9

6 Waveform **10**

7 Đánh giá **10**
7.1 Tính đúng đắn . . . 10
7.2 Hiệu năng . . . 10

8 Kết luận **10**

A Các lệnh mô phỏng **11**

Tóm tắt

Báo cáo trình bày quá trình hiện thực và kiểm thử bộ xử lý RISC-V RV32I single-cycle cho Milestone 2. Thiết kế được tích hợp vào template chính thức của môn học, giữ nguyên wrapper, testbench và các chương trình kiểm thử do TA cung cấp. Kết quả Cadence Xcelium đạt 37/37 instruction tests, 86/86 check groups, 404988/404988 cases và ISA score 6.00/6.00. Chương trình Tower of Hanoi cũng chạy đúng ở cả hai định dạng instruction memory 1B và 4B.

1 Mục tiêu và yêu cầu

Milestone 2 yêu cầu thiết kế một processor RV32I single-cycle. Mỗi instruction phải hoàn thành trong một chu kỳ, bao gồm instruction fetch, decode, execute, memory access và write-back. Các mục tiêu chính:

- hiện thực datapath và controller tương thích RV32I;
- hỗ trợ ALU, register file, branch, jump và load/store;
- giữ đúng tên module và interface trong template;
- hỗ trợ IMEM byte-addressed, little-endian;
- kiểm thử với ISA regression và các chương trình C;
- đạt IPC xấp xỉ 1.

Theo specification, các extension `M`, `F`, `D`, `A` và `C` không thuộc phạm vi của milestone này.

2 Cấu trúc project và môi trường

Project được triển khai trong template:

Thư mục	Chức năng
`00_src/`	RTL của processor và memory
`01_bench/`	Testbench chính thức của TA
`02_test/`	ISA test và các chương trình C
`03_sim/`	Makefile và filelist simulation
`90_doc/`	Báo cáo và tài liệu nộp

Mô phỏng cuối được chạy trên compute node bằng Cadence Xcelium:

```
module load cadence/xcelium/2409
make filelist
make sim TEST=ISA_TEST IMEM_FORMAT=1B WAVE=NONE
```

Wrapper `milestone2.sv`, testbench `tb_milestone2.sv`, các file trong `01_bench/`, các binary trong `02_test/` và Makefile của TA được giữ nguyên.

3 Kiến trúc processor

3.1 Tổng quan kiến trúc single-cycle

Processor được xây dựng theo kiến trúc single-cycle, trong đó mỗi instruction hoàn thành toàn bộ các giai đoạn fetch, decode, execute, memory access và write-back trong một chu kỳ clock. Program counter được reset về `0x80000000`. Từ địa chỉ này, instruction hiện tại được đọc từ IMEM và đưa vào các khối controller, immediate generator, register file, BRC và ALU.

Các khối trong datapath hoạt động phối hợp trong cùng một chu kỳ. Controller giải mã instruction và tạo tín hiệu điều khiển; register file cung cấp các toán hạng; ALU thực hiện phép toán hoặc tính địa chỉ; LSU định dạng dữ liệu load/store; cuối cùng write-back ghi kết quả về register file nếu instruction yêu cầu.

Next PC được chọn giữa `PC+4` và địa chỉ đích do ALU tạo ra. Với JALR, bit thấp nhất của địa chỉ được xóa:

$$\texttt{next_pc} = \{\texttt{alu_data[31:1]}, 1'b0\}.$$

3.2 Sơ đồ khối và đường đi dữ liệu

3.2.1 Đường đi dữ liệu trong một chu kỳ

Đường đi dữ liệu bắt đầu từ PC và kết thúc ở write-back hoặc cập nhật PC cho chu kỳ tiếp theo. Trình tự xử lý có thể tóm tắt như sau:

1. PC cung cấp địa chỉ cho `imem.sv` để đọc instruction;
2. controller giải mã instruction, đồng thời `immgen.sv` trích xuất immediate;
3. register file đọc hai toán hạng nguồn;
4. BRC kiểm tra điều kiện branch và ALU thực hiện phép toán hoặc tính địa chỉ;
5. LSU định dạng dữ liệu và kết nối datapath với `dmem.sv` khi instruction là load/store;
6. bộ chọn write-back chọn dữ liệu từ ALU, DMEM, PC+4 hoặc immediate để ghi vào thanh ghi đích;
7. mạch next-PC chọn `PC+4` hoặc địa chỉ branch/jump.

Do các bước trên thuộc cùng một đường combinational giữa hai cạnh clock, processor có thể hoàn thành một instruction trong mỗi chu kỳ.

3.3 Các khối chức năng chính

3.3.1 ALU và register file

Register file có 32 thanh ghi 32-bit, hai cổng đọc bất đồng bộ và một cổng ghi đồng bộ. Thanh ghi `x0` luôn đọc được giá trị zero. ALU hỗ trợ cộng, trừ, logic, shift và so sánh signed/unsigned cần thiết cho RV32I.

3.3.2 Branch comparison unit

BRC tạo hai kết quả `equal` và `less`. Controller sử dụng `i_br_unsigned` để chọn signed hoặc unsigned comparison. Các branch được hỗ trợ là `BEQ`, `BNE`, `BLT`, `BGE`, `BLTU` và `BGEU`.

3.3.3 Controller và immediate generator

Controller sử dụng các trường `opcode`, `funct3` và `funct7` để xác định loại instruction, phép toán ALU, nguồn toán hạng, loại immediate, quyền truy cập memory và nguồn write-back. Đối với instruction hợp lệ, controller tạo các tín hiệu điều khiển tương ứng; đối với instruction không hợp lệ, các tín hiệu ghi được khóa.

Immediate generator hỗ trợ các dạng U, J, I, B và S. Immediate B-type và J-type được sắp xếp lại đúng vị trí bit, bit 0 bằng zero, sau đó sign-extend thành 32 bit.

3.3.4 Load/store unit

LSU tạo địa chỉ byte, dữ liệu store và byte mask cho `SB`, `SH`, `SW`. Với load, LSU chọn đúng byte hoặc halfword theo địa chỉ thấp và thực hiện sign-extension/zero-extension cho `LB`, `LH`, `LW`, `LBU`, `LHU`.

3.3.5 Instruction memory và data memory

IMEM có asynchronous read và little-endian byte order. Hai định dạng program image được hỗ trợ:

- **1B**: mỗi dòng file chứa một byte;
- **4B**: mỗi dòng file chứa một word 32-bit.

Định dạng 4B được chọn bởi macro `IMEM_4B`. Word index dùng:

`i_addr[ADDR_WIDTH-1:2]`.

DMEM là byte-addressed, đọc bất đồng bộ và ghi đồng bộ theo byte mask. Memory map sử dụng trong wrapper:

Bảng 1: Memory map của Milestone 2

Mode	IMEM	DMEM
C program	`0x8000_0000-0x8007_FFFF`	`0x8008_0000-0x8008_FFFF`
ISA test	`0x8000_0000-0x83FF_FFFF`	`0x8400_0000-0x8400_FFFF`

3.4 Tổ chức mã nguồn RTL

3.4.1 Tổ chức mã nguồn

Mã nguồn được tổ chức theo từng khối chức năng thay vì đặt toàn bộ thiết kế trong một module lớn. Cách tổ chức này làm cho sơ đồ thiết kế dễ theo dõi hơn, đồng thời cho phép kiểm tra từng khối trước khi ghép thành processor single-cycle hoàn chỉnh. Module trung tâm là `singlecycle.sv`; các module còn lại lần lượt cung cấp chức năng giải mã, tính toán, truy cập bộ nhớ và ghi kết quả.

Bảng 2: Tổ chức các file RTL trong thư mục `00_src/`

File	Chức năng
`singlecycle.sv`	Tích hợp PC, datapath, controller, register file, ALU, BRC, LSU và write-back.
`controller.sv`, `immgen.sv`	Giải mã instruction, kiểm tra tính hợp lệ và tạo tín hiệu điều khiển/immediate.
`adder.sv`, `alu.sv`, `comparator.sv`, `brc.sv`, `shifter.sv`	Các khối tính toán và so sánh được datapath sử dụng.
`regfile.sv`, `lsu.sv`	Register file và logic định dạng load/store.
`imem.sv`, `dmem.sv`	Instruction memory và data memory.
`milestone2.sv`	Wrapper kết nối core với memory, MMIO và testbench.

3.4.2 Điều phối và kiểm tra tính hợp lệ

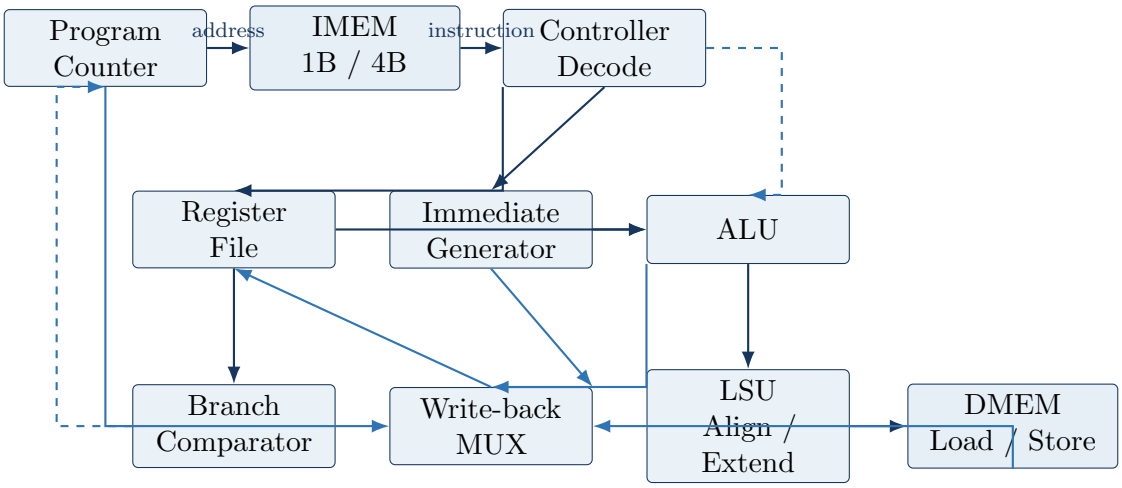
Trong mã nguồn, controller là khối điều phối trung tâm. Từ các trường opcode, funct3 và funct7, controller xác định loại instruction, chọn nguồn toán hạng, phép toán ALU, loại immediate và nguồn write-back. Một số tín hiệu điều khiển quan trọng là:

- `o_pc_sel`: chọn địa chỉ PC kế tiếp;
- `o_rd_wren`: cho phép ghi register file;
- `o_imm_sel`: chọn loại immediate;
- `o_opa_sel`, `o_opb_sel`: chọn toán hạng ALU;
- `o_alu_op`: chọn phép toán ALU;
- `o_mem_wren`: tạo yêu cầu store;
- `o_wb_sel`: chọn nguồn dữ liệu write-back;
- `o_instr_valid`: báo instruction hợp lệ cho testbench.

Đối với instruction không hợp lệ, controller không tạo yêu cầu ghi register file hoặc data memory. Trong thời gian reset, các tín hiệu ghi cũng được khóa và `o_instr_valid` được giữ ở mức thấp. Nhờ đó, testbench chỉ đếm những instruction thực sự hoàn thành.

3.4.3 Sơ đồ khối tổng thể

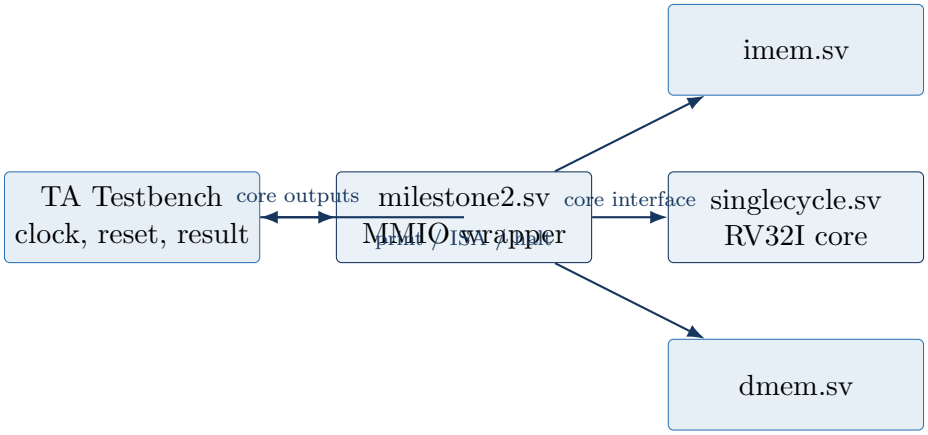
Sơ đồ ở Hình 1 mô tả mối liên hệ giữa các khối chính của processor. Đường đi bắt đầu từ PC và IMEM, sau đó instruction được phân tích bởi controller. Hai toán hạng từ register file đi vào BRC và ALU. Kết quả ALU có thể được chuyển đến LSU để truy cập DMEM, được đưa qua write-back để ghi vào thanh ghi đích, hoặc được dùng làm địa chỉ branch/jump cho PC kế tiếp. Vì tất cả các khối này hoạt động trong cùng một chu kỳ, sơ đồ thể hiện đúng đặc trưng của kiến trúc single-cycle.



Hình 1: Sơ đồ khối datapath và control của processor single-cycle.

3.4.4 Sơ đồ wrapper và giao tiếp MMIO

Wrapper `milestone2.sv` nằm bên ngoài core `singlecycle.sv`, vì vậy phần infrastructure của TA không cần biết chi tiết datapath bên trong. Wrapper kết nối instruction bus và data bus với IMEM/DMEM. Ngoài ra, wrapper nhận diện một số địa chỉ store đặc biệt và chuyển chúng thành các tín hiệu dễ quan sát cho testbench: in ký tự, gửi kết quả ISA và báo hiệu chương trình đã kết thúc.



Hình 2: Wrapper, memory và giao tiếp với testbench chính thức.

3.4.5 Interface của core

Interface của `singlecycle.sv` được giữ nguyên theo template TA:

```
input  i_clk, i_rstn
output o_db_valid
output o_imem_addr
input  i_imem_rdata
output o_dmem_addr, o_dmem_wdata
output o_dmem_bmsk, o_dmem_wren
input  i_dmem_rdata
```

Đây là ranh giới giữa core và phần kiểm thử. Nhờ giữ đúng interface, core mới có thể được thay vào template và chạy trực tiếp với wrapper, testbench và các chương trình chính thức mà không phải chỉnh sửa các file infrastructure của môn học.

3.5 Testbench và phương pháp kiểm tra

Testbench chính thức `tb_milestone2.sv` nhận các tín hiệu quan sát từ wrapper và dùng chúng để kiểm tra chương trình. Testbench đếm cycle và instruction thông qua `o_db_valid`. Với N_{instr} instruction hợp lệ trong N_{cycle} cycle:

$$\text{IPC} = \frac{N_{\text{instr}}}{N_{\text{cycle}}}.$$

Như vậy, phần RTL của sinh viên chỉ cần tuân thủ interface của `singlecycle.sv`; việc kiểm tra kết quả, đếm instruction và kết thúc mô phỏng được thực hiện bởi infrastructure của TA.

4 Quy trình simulation

Các lệnh được dùng trên remote server:

```
make filelist
make sim TEST=ISA_TEST IMEM_FORMAT=1B WAVE=NONE
make sim TEST=ISA_TEST IMEM_FORMAT=4B WAVE=NONE
make sim TEST=PROG_HANOI IMEM_FORMAT=1B WAVE=NONE
make sim TEST=PROG_HANOI IMEM_FORMAT=4B WAVE=NONE
```

Khi cần waveform:

```
make sim TEST=ISA_TEST WAVE=SHM \
  WAVE_START_TIME=8400000 WAVE_STOP_TIME=8500000
```

5 Kết quả simulation

5.1 ISA regression

ISA_TEST được chạy bằng Xcelium ở cả IMEM 1B và 4B. Kết quả trích từ console:

```
Total tests passed   : 37/37
Check groups passed  : 86/86
Total cases passed   : 404988/404988
Total cycles         : 5497867
Total instructions   : 5497867
IPC                  : 1.0000
ISA score            : 6.00/6.00
```

Bảng 3: Tóm tắt ISA regression

Chỉ số	Kết quả	Trạng thái
Instruction tests	37/37	PASS
Check groups	86/86	PASS
Individual cases	404988/404988	PASS
IPC	1.0000	PASS
ISA score	6.00/6.00	PASS

5.2 Tower of Hanoi

Kết quả của `PROG_HANOI` ở cả 1B và 4B:

```
Disks: 12
Total moves:     4095 (expected 4095)
Move checksum:   0xd25f5c74 (expected 0xd25f5c74)
Legality errors: 0
Final result:    PASS

PERFORMANCE SUMMARY
Total cycles         : 339657
Total instructions   : 339657
IPC                  : 1.0000
```

Bảng 4: Kết quả Hanoi

IMEM	Moves	Checksum	Errors	Kết quả
1B	4095/4095	`0xd25f5c74`	0	PASS
4B	4095/4095	`0xd25f5c74`	0	PASS

Các chương trình C còn lại. Template cung cấp thêm `PROG_SORT`, `PROG_MATRIX`, `PROG_SOFTFLOAT`, `PROG_SIGMOID` và `PROG_CONVOLUTION`. Các chương trình được chạy bằng Cadence Xcelium với `WAVE=NONE`. Mỗi chương trình được chạy ở cả hai định dạng IMEM để kiểm tra rằng cách đóng gói instruction image không làm thay đổi kết quả thực thi.

5.3 PROG_SORT: Bubble sort và các thuật toán sắp xếp

`PROG_SORT` sử dụng một mảng signed integer gồm 42 phần tử. Chương trình thực hiện và so sánh sáu thuật toán:

1. Bubble Sort;
2. Insertion Sort;
3. Selection Sort;
4. Quick Sort;
5. Merge Sort;

6. Heap Sort.

Cả sáu thuật toán đều trả về cùng một dãy tăng dần và đều PASS. Kết quả quan trọng của console là:

```
1. Bubble Sort:   ==> Result: PASS
2. Insertion Sort: ==> Result: PASS
3. Selection Sort: ==> Result: PASS
4. Quick Sort:     ==> Result: PASS
5. Merge Sort:     ==> Result: PASS
6. Heap Sort:      ==> Result: PASS

Total cycles         : 486758
Total instructions   : 486758
IPC                  : 1.0000
```

Kết quả 1B và 4B giống nhau: cả sáu thuật toán đều PASS, với 486758 cycles và IPC bằng 1.0000.

5.4 PROG_MATRIX: Nhân ma trận

`PROG_MATRIX` kiểm tra phép nhân hai ma trận signed kích thước 8×8. Ngoài việc kiểm tra trực tiếp kết quả, chương trình dùng quan hệ:

$$\text{transpose}(A)\,\text{transpose}(B) = \text{transpose}(BA)$$

để cross-check kết quả độc lập.

Kết quả từ console:

```
Checksum:          0xaae2cc7e (expected 0xaae2cc7e)
Cross-check errors: 0
Final result:      PASS

Total cycles         : 171393
Total instructions   : 171393
IPC                  : 1.0000
```

Hai định dạng 1B và 4B đều cho cùng checksum, không có lỗi cross-check và cùng IPC bằng 1.0000.

5.5 PROG_SOFTFLOAT: Software floating-point

`PROG_SOFTFLOAT` kiểm tra số học floating-point bằng phần mềm trên RV32I. Chương trình kiểm tra các định dạng binary16, bfloat16, binary32 và binary64, bao gồm cộng, trừ, nhân, chia và căn bậc hai. Các trường hợp biên như overflow và subnormal cũng được kiểm tra.

Kết quả self-check:

```
Self-check: 56/56  PASS

Total cycles         : 2857414
Total instructions   : 2857414
IPC                  : 1.0000
```

Đây là software floating-point, không phải phần cứng extension `F` hoặc `D`. Kết quả 1B và 4B đều đạt 56/56 trường hợp PASS, cho thấy processor RV32I có thể thực thi chương trình số học nhiều bước và nhiều phép load/store mà không cần hardware floating-point extension.

5.6 PROG_SIGMOID: Sigmoid và classifier

`PROG_SIGMOID` đánh giá hàm sigmoid fixed-point/software double-precision trên nhiều giá trị đầu vào, sau đó dùng các giá trị sigmoid làm feature cho classifier ba lớp. Chương trình kiểm tra cả đường cong sigmoid và nhãn phân loại.

Kết quả:

```
Self-check: 62/62  PASS

Total cycles         : 3822619
Total instructions   : 3822619
IPC                  : 1.0000
```

Các mẫu classifier được phân loại thành class 0, class 1, class 2 và class 1 tương ứng với bốn input mẫu. Cả hai định dạng IMEM đều đạt 62/62 PASS và IPC bằng 1.0000.

5.7 PROG_CONVOLUTION: Convolution hai chiều

`PROG_CONVOLUTION` thực hiện convolution hai chiều trên ma trận đầu vào và áp dụng các kernel Gaussian và Sobel. Chương trình cũng tính độ lớn biên theo:

$$|X| + |Y|.$$

Kết quả self-check:

```
Self-check: 125/125  PASS

Total cycles         : 1811378
Total instructions   : 1811378
IPC                  : 1.0000
```

Kết quả 1B và 4B đều đạt 125/125 PASS, không có sai khác về số cycle hoặc IPC.

5.8 Tổng hợp kết quả các chương trình C

Bảng 5: Tổng hợp kết quả simulation của các chương trình C

Chương trình	Self-check	Cycles	IPC	1B/4B
`PROG_HANOI`	PASS	339657	1.0000	PASS/PASS
`PROG_SORT`	6/6 PASS	486758	1.0000	PASS/PASS
`PROG_MATRIX`	PASS	171393	1.0000	PASS/PASS
`PROG_SOFTFLOAT`	56/56 PASS	2857414	1.0000	PASS/PASS
`PROG_SIGMOID`	62/62 PASS	3822619	1.0000	PASS/PASS
`PROG_CONVOLUTION`	125/125 PASS	1811378	1.0000	PASS/PASS

Tất cả chương trình đều kết thúc bằng `$finish` từ testbench, không bị timeout và có số instruction bằng số cycle. Điều này xác nhận cả chức năng tính toán, giao tiếp MMIO, cơ chế halt và instruction-memory format selection đều hoạt động đúng.

6 Waveform

Waveform được tạo dưới dạng SHM bằng Xcelium. Hình nên thể hiện tối thiểu PC, instruction, `o_db_valid`, địa chỉ DMEM, dữ liệu store, byte mask và write enable.

TODO: Chèn ảnh waveform SimVision tại đây. Có thể đặt ảnh trong `90_doc/Figures/` rồi dùng `\includegraphics`.

Hình 3: Vị trí chèn waveform của processor.

7 Đánh giá

7.1 Tính đúng đắn

ISA regression xác nhận các nhóm chức năng immediate, ALU, shift, comparison, jump, branch, load/store và mở rộng dữ liệu khi load. Việc cùng chương trình Hanoi PASS ở hai định dạng chứng minh IMEM format selection và word indexing hoạt động nhất quán.

7.2 Hiệu năng

ISA test và Hanoi đều có IPC bằng 1.0000. Đây là kết quả phù hợp với single-cycle architecture, trong đó mỗi instruction hợp lệ hoàn thành trong một cycle.

8 Kết luận

Processor RV32I single-cycle đã được tích hợp vào template chính thức của TA mà không thay đổi wrapper hoặc testbench. ISA_TEST đạt 6.00/6.00, và Tower of Hanoi PASS ở cả IMEM 1B và 4B. Báo cáo sẽ được hoàn thiện bằng console log đầy đủ và waveform sau khi chụp/export từ server.

A Các lệnh mô phỏng

```
make sim TEST=ISA_TEST IMEM_FORMAT=1B WAVE=NONE
make sim TEST=ISA_TEST IMEM_FORMAT=4B WAVE=NONE
make sim TEST=PROG_HANOI IMEM_FORMAT=1B WAVE=NONE
make sim TEST=PROG_HANOI IMEM_FORMAT=4B WAVE=NONE
make sim TEST=ISA_TEST WAVE=SHM \
  WAVE_START_TIME=8400000 WAVE_STOP_TIME=8500000
```